

MAJOR PROJECT REPORT

on

SECURE LOGIN SYSTEM

WITH

PASSWORD HASHING

Cyber Security Domain

Submitted as a part of

Cyber Security Internship

Submitted By:
Anuja Umesh Shetty

Organization:
Topgrade Innovation

Technology & Tools Used:
Python | Tkinter | bcrypt | File Handling
| Visual Studio Code

Academic Year
2026-27

1. PROBLEM STATEMENT

User authentication systems handle sensitive information such as usernames and passwords and must protect them from unauthorized access. Storing passwords in plain text can expose user credentials and create serious security risks if the stored data is compromised.

This project focuses on developing a **Secure Login System** that protects user passwords using **bcrypt hashing**. The system provides user registration, secure password storage, login validation, logout functionality, duplicate username prevention, and account lockout after repeated failed login attempts.

2. OBJECTIVE

The main objectives of the project are:

- To develop a secure user registration and login system.
- To protect passwords using **bcrypt hashing**.
- To securely store user credentials using file handling.
- To validate user credentials during login.
- To provide registration, login, and logout functionality.
- To prevent duplicate username registration.
- To implement account lockout after three unsuccessful login attempts.
- To understand basic cybersecurity concepts related to authentication and password protection.

3. TOOLS & TECHNOLOGIES USED

Python-

Python is used as the primary programming language for developing the Secure Login System. It handles user registration, login validation, password processing, file operations, and account lockout logic.

Tkinter-

Tkinter is used to create the graphical user interface (GUI). It provides the username and password fields, buttons for registration, login and logout, status messages, and result notifications.

Bcrypt-

bcrypt is used for **secure password hashing**. Instead of storing the actual password, the system stores its bcrypt hash in the users.txt file. During login, the entered password is checked against the stored hash.

File Handling-

File handling is used to store registered usernames and their corresponding password hashes in the users.txt file.

Visual Studio Code-

Visual Studio Code is used as the development environment for writing, editing, running, and testing the Python project.

Technologies Summary

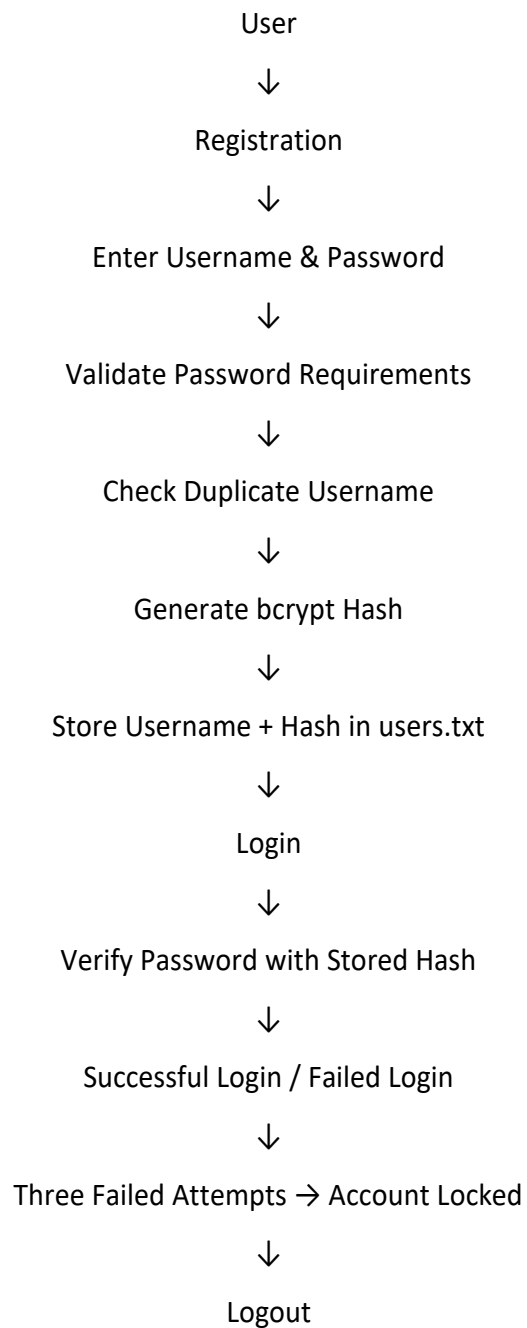
Technology / Tool	Purpose
Python	Application development and logic
Tkinter	Graphical User Interface
bcrypt	Password hashing and verification
File Handling	Secure credential storage
Visual Studio Code	Development and testing

4. SYSTEM DESIGN / WORKING

The Secure Login System follows a simple authentication workflow. The user first registers an account by providing a username and password. The password is validated according to the defined security requirements and then converted into a bcrypt hash before being stored.

During login, the entered username is searched in the stored user records. The entered password is verified against the stored bcrypt hash. If the credentials are correct, the user is authenticated successfully. If an incorrect password is entered repeatedly, the system locks the account after three failed attempts.

System Workflow



Working of the System

1. User Registration:

The user enters a username and password through the graphical interface. The system checks whether the password satisfies the required security conditions and whether the username already exists.

2. Password Hashing:

After successful validation, the password is processed using bcrypt. Only the generated hash is stored in the users.txt file.

3. Login Authentication:

During login, the system retrieves the stored hash and uses bcrypt to verify the entered password.

4. Failed Login Protection:

The system tracks unsuccessful login attempts. After three incorrect password attempts, the account is locked.

5. Logout:

After successful authentication, the user can log out of the system. The login status is then changed to logged out.

5. CODE EXPLANATION

The project is implemented using Python. The main program contains separate functions for registration, login, and logout, along with password validation, bcrypt hashing, file handling, and account lockout logic.

1. Registration Module

The registration function accepts a username and password from the user. It checks whether both fields are filled and validates the password according to the required security conditions. The system also checks whether the username already exists. If the username is available, the password is converted into a bcrypt hash and the username and hash are stored in the users.txt file.

2. Password Hashing

The password is hashed using the bcrypt library before storage. The actual password is never written to the users.txt file.

The hashing process can be represented as:

User Password → bcrypt Hashing → Hashed Password → users.txt

This protects the password from being directly exposed if the storage file is accessed.

3. Login Module

The login function reads the stored username and password hash from users.txt. When the user enters a password, bcrypt verifies it against the stored hash.

- Correct password → **Login Successful**
- Incorrect password → **Login Failed**
- Username not found → **Username Not Found**

4. Account Lockout

The system maintains a count of unsuccessful login attempts. After **three incorrect password attempts**, the account is locked and further login attempts are prevented.

This provides additional protection against repeated password-guessing attempts.

5. Logout Module

The logout function terminates the current login session and changes the application status to **Logged out**. The username and password fields are also cleared.

6. Graphical User Interface

Tkinter is used to provide the graphical interface. It contains:

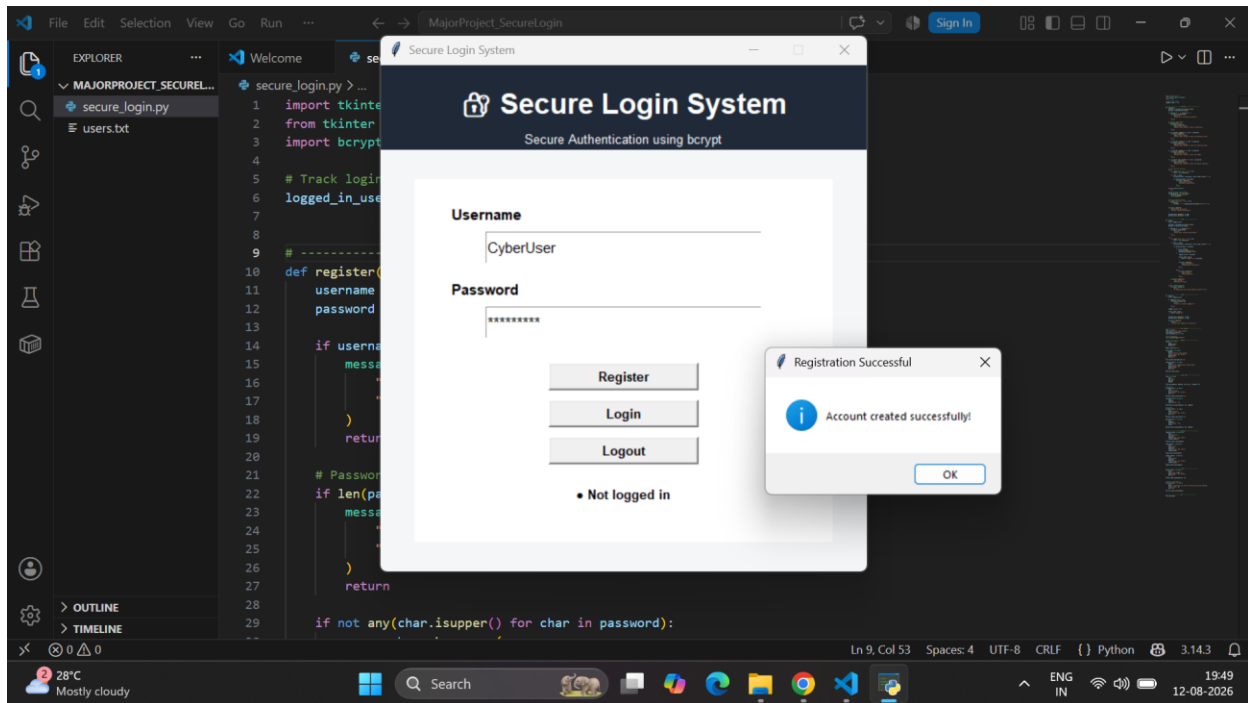
- Username input field
- Password input field
- Register button
- Login button
- Logout button
- Login status display
- Security information message

6. PROJECT OUTPUT

The following screenshots demonstrate the successful execution and major features of the Secure Login System.

6.1 Registration Successful

The registration module successfully creates a new user account after validating the username and password requirements.

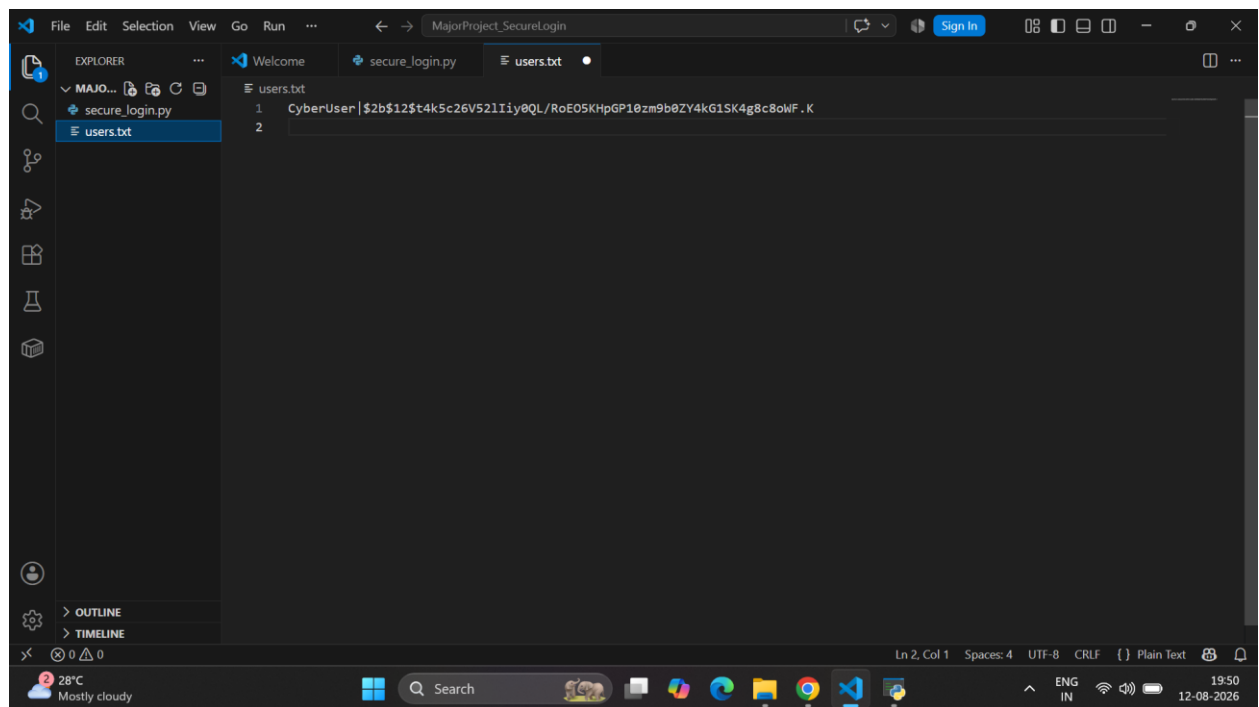


Output: The system displays a confirmation message indicating that the account has been created successfully.

6.2 Secure Password Storage

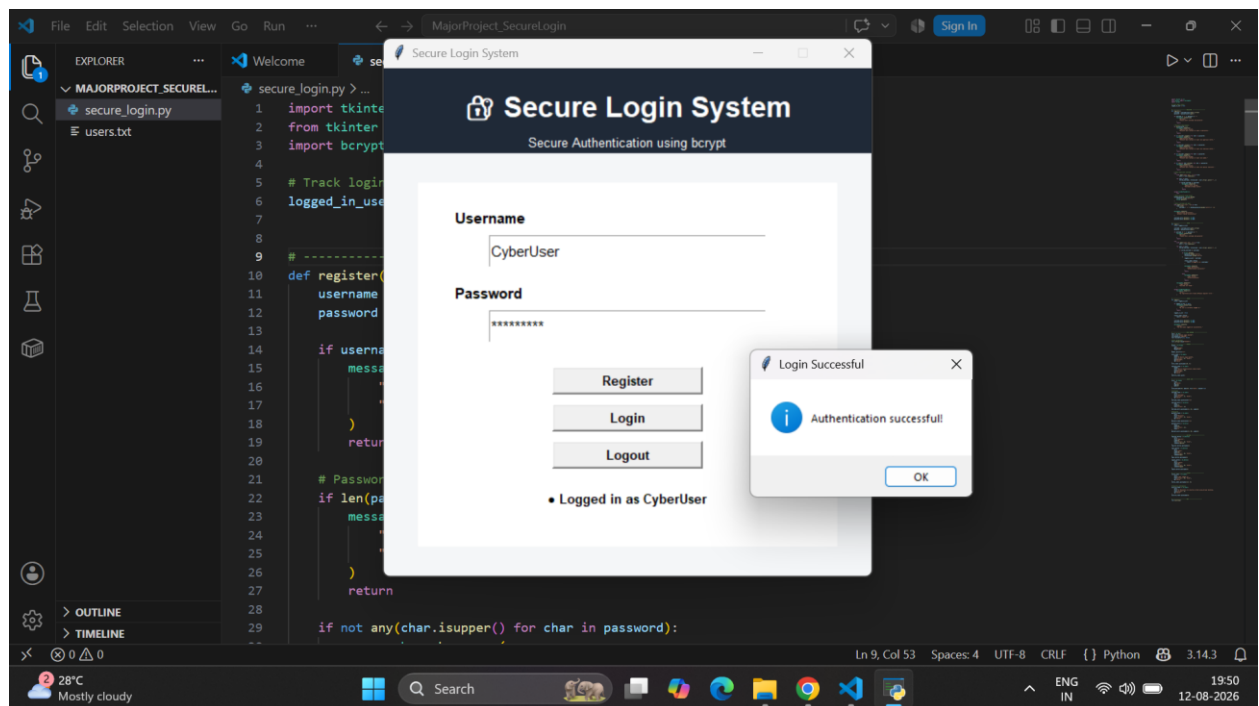
The registered username and its bcrypt-generated password hash are stored in the users.txt file. The original password is not stored in plain text.

Output: The file contains the username followed by the bcrypt hash.



6.3 Login Successful

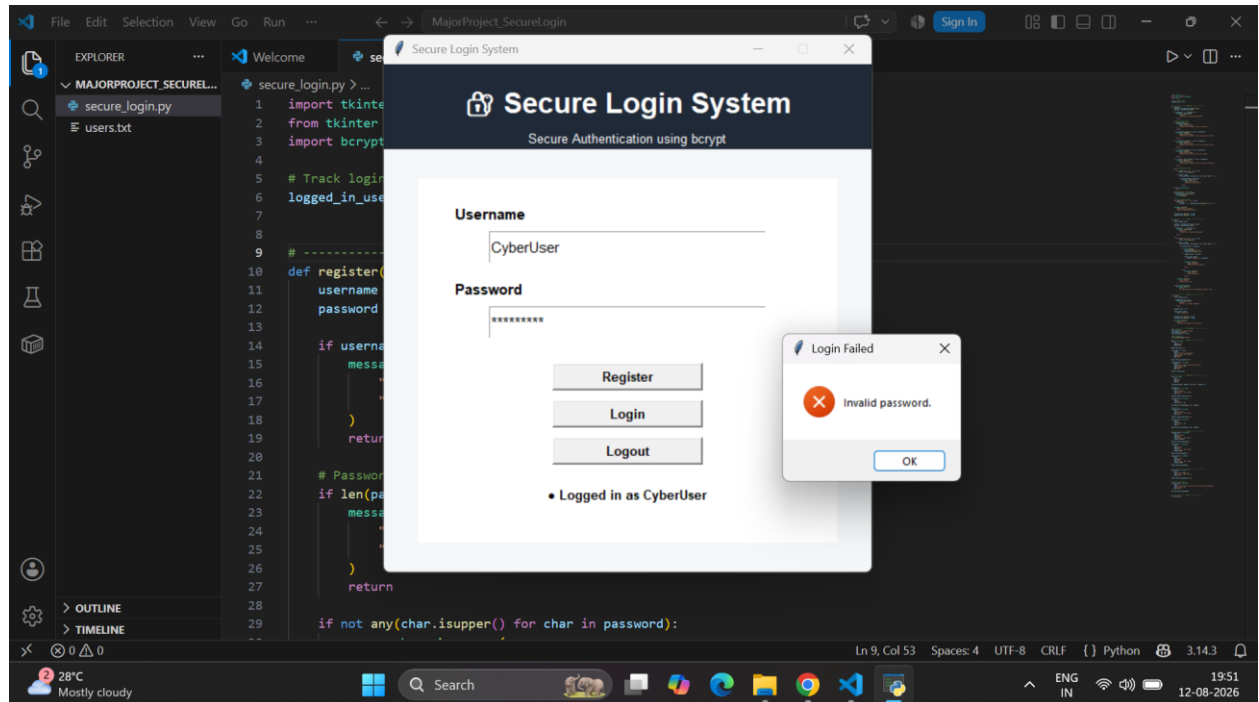
The system verifies the entered password against the stored bcrypt hash and grants access when the credentials are valid.



Output: The system displays an authentication success message and updates the login status.

6.4 Login Failure

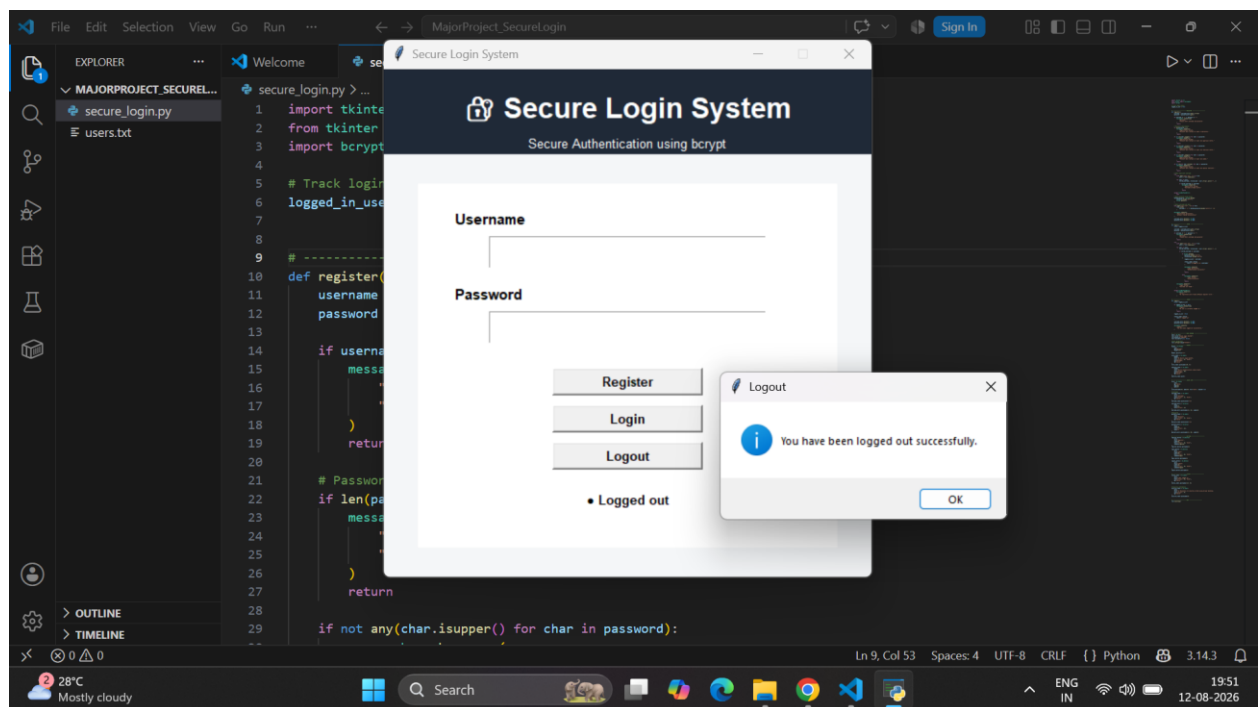
The system rejects incorrect passwords and displays an appropriate error message.



Output: The system displays an invalid-password message.

6.5 Logout

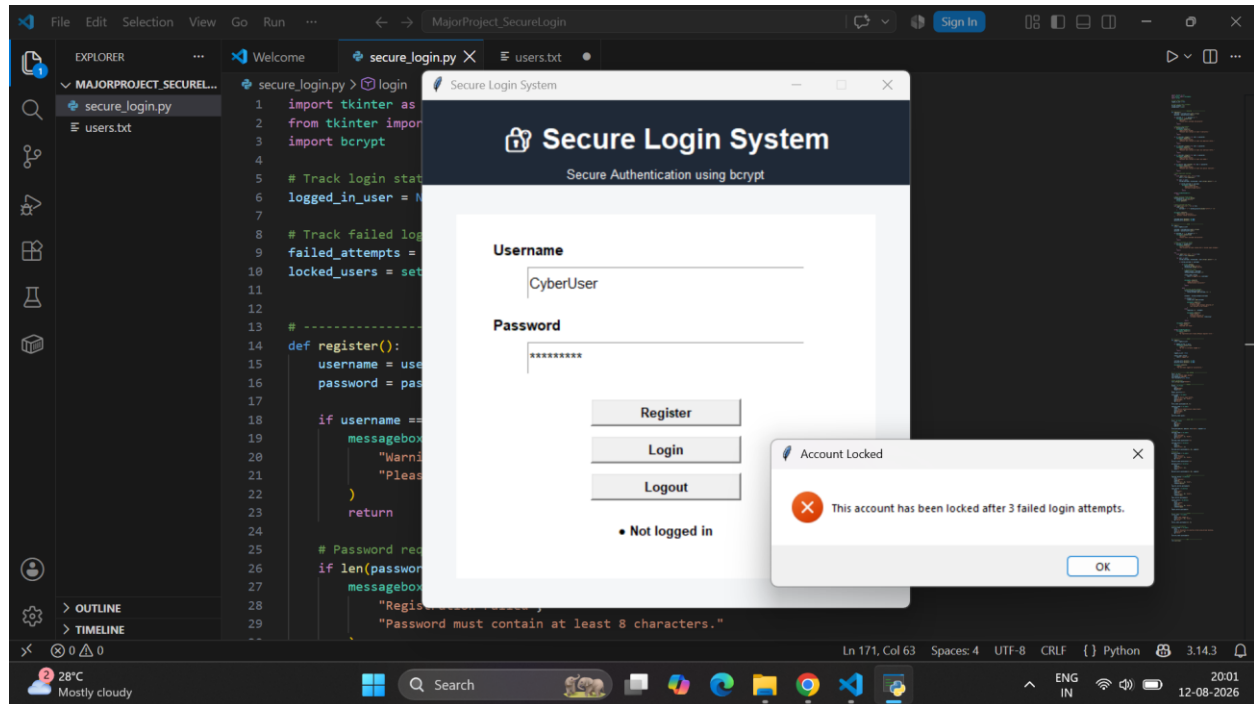
The user can securely end the current session using the Logout option.



Output: The system confirms that the user has been logged out and updates the status.

6.6 Account Lockout

After three unsuccessful login attempts, the system locks the account to prevent repeated password-guessing attempts.



Output: The system displays an account-locked message.

7. OUTPUT DESCRIPTION

The Secure Login System was successfully implemented and tested using different user authentication scenarios.

- Registration Output

The system successfully registers a new user after validating the username and password. Duplicate usernames are rejected to maintain unique user accounts.

- Password Storage Output

After registration, the password is converted into a bcrypt hash before being stored in users.txt. This ensures that the original password is not stored as plain text.

- Login Output

The system successfully authenticates users when the correct username and password are provided. Incorrect passwords are rejected with an appropriate error message.

- Logout Output

After successful authentication, the user can log out using the Logout button. The system clears the login information and updates the status to indicate that the user is logged out.

- Account Lockout Output

The system tracks unsuccessful login attempts and locks the account after three consecutive incorrect password attempts. This provides an additional security measure against repeated password-guessing attempts.

Overall Result

The project successfully demonstrates the basic working of a secure authentication system with **password hashing, credential verification, session logout, duplicate username prevention, and account lockout.**

8. CONCLUSION

The **Secure Login System with Password Hashing** was successfully developed using Python, Tkinter, bcrypt, and file handling. The project demonstrates secure password storage, user authentication, logout functionality, duplicate username prevention, and account lockout after repeated failed login attempts.

The project helped in understanding the importance of **secure authentication and password protection** in cybersecurity applications. The implemented system successfully performed the required operations and produced the expected results during testing.

9. FUTURE SCOPE

The Secure Login System can be further improved by adding advanced security and authentication features. Possible future enhancements include:

- Integration with a **database** for managing user accounts.
- Implementation of **multi-factor authentication (MFA)** using OTP or authenticator applications.
- Addition of **password reset and account recovery** functionality.
- Temporary lockout with automatic account unlocking after a specified period.
- Improved password policies and password-strength analysis.

- Secure web-based deployment with **HTTPS**.
- Addition of login activity and security event logging.
- Role-based access control for different types of users.

These enhancements can make the system more scalable, secure, and suitable for real-world authentication applications.